Symphony and Beethoven Taxonomy Manual

Oct 15, 2026 · @f

1. What this manual is for

This manual is how to use the spec: to run the deep research that checks and extends it, to place any other tool in the same taxonomy, and to explain the approach to grant reviewers, engineers and lab leads. The spec says what is true and how sure we are; the manual says what to do with it.

The two travel together. The spec holds the taxonomy, the placement, the graded claims and the open research questions. This manual holds the procedures, the prompts to paste, the worksheet and the talking points.

A finding changes the spec first, then every talking point that rests on it. The research notes behind both are in the project folder under research/taxonomy/notes/, one file per topic, each line with its source.

2. Before you start

Read, in this order (about twenty minutes): the spec's section 9 (the Actions story), section 8 (what is unique), section 7 (the need), then section 10 (the risks). Sections 3 to 5 are reference.

Already checked against primary sources on 2026-10-15, so the deep research need not redo them:

- GitHub Actions billing, runner sizes and limits, read from the github/docs repository at commit 12691d2
- The text of GitHub's Actions terms, effective 2026-04-27
- Anthropic's pages on Claude Code GitHub Actions, costs, agent teams, Remote Control and legal use, and claude.com/pricing
- gh-aw's work queue and its refusal of the subscription token, from its repository at dc91dbed
- Copilot automations not being available in public repositories, and Copilot's cloud agent limits
- CooperBench's counts, from the paper's PDF in the authors' repository

- How the claim behaves under git, by experiment on local repositories (spec, risks 12 to 14)
- A search for a rival that combines Symphony's mechanisms (spec, section 8)

Still open are the ten research questions in the spec's section 11. Three of them need measurements, not reading: R1 (the benchmark run), R3 (cost of a coordination step) and R4 (the claim on github.com). Those run in their own threads, by your choice, because they spend tokens or touch live repositories.

What this project's sessions cannot reach. The cloud environment blocks docs.github.com, github.blog, arxiv.org, metr.org, cursor.com, gartner.com, thoughtworks.com and openspineconsortium.com, and the first research pass used up its web-search allowance for the turn. GitHub's docs were read from the github/docs repository instead, and papers from their authors' repositories where there was one, graded Verified, or else from third-party mirrors and search summaries, graded Reported. That is why the deep research below runs in claude.ai's Research mode, which reaches the open web, and why its report comes back here to be folded into the spec.

Two holds stay in force. Data from the coordination experiment stays out of the grant until you say so. Nothing in either document is a legal opinion: the terms questions (R5, R6) are for GitHub, Anthropic or a lawyer to answer.

3. Running the deep research

Deep research is something you start; a thread here cannot start it for you. It runs in three passes.

Pass 1: the open web, in claude.ai Research

1. Open claude.ai/new.
2. Below the message box, turn on **Research** (and web search, if it is a separate switch).
3. Paste the message below as it is, and send it.
4. When the report is done, copy its whole text.

I am placing two pieces of software in a taxonomy of agent software and need a deep, sourced check of the claims below, as of today. Prefer primary sources (vendor docs and terms pages, papers, repositories); mark anything taken from a secondary source as Reported. For every claim give a verdict (Confirmed, Partly true, Refuted or Unverifiable), the evidence with links, dates and short quotes, and corrected wording. Start the report with a table of verdicts, and end it with a list of what you could not verify.

The systems:
- Symphony (github.com/OpenSpineConsortium/symphony-protocol, MIT): a coordination protocol for Claude Code agents kept in a GitHub repository. Issues are the queue. A worker claims issue N by pushing a new branch claude/N whose first commit adds a claim file with the plan and the files it will touch; git's compare-and-swap on ref creation lets exactly one of two racing creations win. GitHub Actions jobs render a board (a file on a branch), release a claim idle for 24 hours while keeping its work under a renamed branch, and order work that touches the same files. Workers are people's own Claude Code sessions and agents running as GitHub Actions jobs through anthropics/claude-code-action on a Claude subscription token (CLAUDE_CODE_OAUTH_TOKEN). It has no server of its own.
- Beethoven (private): runs a student's own Claude Code Remote Control session as a Slurm batch job on a university cluster, started through Open OnDemand, and plans gates that enforce research reporting checklists (REFORMS, CLAIM) over each run's record; a required check already holds the code its agents write to contracts checked by CrossHair and Z3.

Claims and questions:
1. GitHub Actions: standard GitHub-hosted runners are free and unlimited on public repositories (4 vCPU, 16 GB on Linux), bounded by concurrency (20 jobs on Free, 40 on Pro, 60 on Team, 500 on Enterprise), 6-hour jobs and GITHUB_TOKEN's 1,000 requests an hour per repository; larger and GPU runners are always billed; private repositories get 2,000, 3,000 or 50,000 included minutes a month, then $0.006 a minute on Linux since 2026-01-01. Report any change since 2026-10-15, and the status of the self-hosted runner charge GitHub announced on 2025-12-16 and withdrew the next day.
2. GitHub's Actions terms (effective 2026-04-27) limit GitHub-hosted runners to activity related to "the software project associated with the repository" and forbid offering Actions or its elements for commercial purposes. How have GitHub, lawyers or practitioners read these clauses for AI coding agents running in Actions, for benchmarks that run other projects' tasks, and for a paid product whose open runtime runs in customers' own Actions?
3. Anthropic: claude-code-action accepts CLAUDE_CODE_OAUTH_TOKEN from claude setup-token, so runs use the subscription instead of API billing; Anthropic

advises an API key for a secret shared across repositories, and its legal page says plan limits assume "ordinary, individual usage". What is Anthropic's current position on several automated agents in CI on one person's subscription? Will claude-code-action keep supporting the token when bare mode becomes the default for claude -p?
4. Uniqueness: is there any system, product or paper that has four of these five, or (a), (c) and (d) together: (a) a claim made atomic by a git ref on the shared forge; (b) automatic release of idle claims by a CI job that keeps the work; (c) a coordinator and board that run as CI jobs, with state only in forge objects; (d) people's interactive sessions and CI agents under one claim rule, on a flat-rate subscription; (e) validation of the shipped protocol code by exhaustive interleavings against an oracle written from its spec, with mutants. Known neighbors to go beyond: GitHub Agentic Workflows (gh-aw) and its October 2026 work queue, unsorry (agenticsnz/unsorry), GitHub Copilot cloud agent and Agent HQ, openai/symphony, CCPM, Beads and Gas Town, paperclip, Orbi, aeon, agentic-setup, Claude Code agent teams and Projects, Anthropic's C-compiler harness, CooperBench's team mode, Claim Plane (arXiv 2608.00947) and Nerveplane (arXiv 2610.07261). Search FSE, ICSE and ASE 2026, arXiv cs.SE and cs.MA, GitHub and practitioner writing.
5. The need: CooperBench (arXiv 2601.13295) reports two agents, each in its own workspace and able only to message the other, passing far less often than one (for GPT-5, 183 against 315 of 652 pairs). Find replications, rebuttals, and any study comparing a claim- or pull-request-gated protocol on a shared remote with one agent at a matched budget.
6. Free CI elsewhere: confirm from primary sources that Travis CI ended unlimited open-source builds (2020), GitLab.com put public projects on quotas (2021), Azure DevOps stopped new public projects (2026) and will make existing ones private (2027), Cirrus CI stopped running jobs (2026-06-01), and what CircleCI gives open source today.
7. Taxonomy: does any published taxonomy of LLM agents or multi-agent systems have an axis for where the compute runs and who pays for it? Use the strongest surveys of 2025 and 2026. Give Gartner's and Thoughtworks' current categories for coding agents, from their own pages.
8. Research automation: does any AI-scientist system or research workflow tool enforce a reporting checklist (REFORMS, CLAIM, TRIPOD+AI, DOME or similar) as a machine-checked gate over an agent's run? How many items does the CLAIM 2024 update have? Does any other project make contracts checked by symbolic execution the required merge gate for code that agents write, and does any tool start Claude Code's Remote Control from a Slurm batch job through Open OnDemand?
9. The name: how crowded is "Symphony" among agent and coding software?

Pass 2: fold the report into the spec, here

Post this in the project chat, with the report's text pasted under it:

```
Here is the deep research report on the Symphony and Beethoven taxonomy.
Update the taxonomy spec from it: change each claim's grade and wording where
the report has better evidence, add new neighbors to sections 4, 5 and 8,
mark research questions answered when their acceptance criteria are met, and
list in the thread what changed and what the report got wrong.
```

Pass 3: the measurements, one thread each, when you choose

- R1, the benchmark: the coordination experiment thread already holds the plan you chose; nothing new to paste.
- R3, the cost of a coordination step: the measurement pass is already queued after the stuck-claim fix in the protocol work.
- R4, the claim on github.com: post "Run research question R4 of the taxonomy spec on the canary repository" in the project chat.

Scoping a pass

One report per pass. If the report is thin on an item, run pass 1 again with that item alone and its question number. If it is confident without sources, ask it for the primary source of each claim it marked Confirmed. Treat anything it cannot link as Reported.

4. Checking a claim

Every claim in the spec carries one of four grades, defined in the spec's section 1: Verified, Inferred, Reported or Open. A grade rises only with new evidence, and falls as soon as a source contradicts it. This section is how to move one.

Testing a claim, in six steps

1. **Write it with its scope.** One sentence, with the version, the plan and the date it holds for. "Free and unlimited" is a claim about standard runners on public repositories as of 2026-10-15, not about Actions in general.
2. **Write what would refute it, before looking for support.** The spec's section 8 gives one refuting observation for each uniqueness claim, and section 11 gives each research question its acceptance criteria.

3. **Look for the refutation first.** Search by mechanism, not by product name, since a rival rarely uses our words: first push wins, lock branch, claims branch, reaper, idle claim, compare-and-swap with agents, CLAUDE_CODE_OAUTH_TOKEN in a workflow.
4. **Run it when it can be run.** A claim about git or the protocol is cheapest to settle on a scratch repository. The claim-lock check (research/taxonomy/verify/02) did this and found or confirmed three things the documents did not say: the server enforces a claim only at its creation, the push guard judges a lease by the session's own directory rather than the clone a git -C command names, and the idle clock is the pusher's (the spec's risks 12 and 13).
5. **Record the search even when it finds nothing:** where you looked, with which terms, on what date. A search that finds nothing makes a claim Inferred, never Verified, and the spec says how far the search went.
6. **When the claim fails, narrow it or drop it.** Change the spec first, then every talking point that rests on it (section 6).

A worked case. An early draft of the placement figure said only Symphony puts its agents under an atomic claim. The adversarial search found three systems that have one: gh-aw's work queue (a compare-and-swap on a ledger branch), unsorry (first push wins on a shared claims branch) and agentic-setup (a create-only branch push). The atomic claim moved to the spec's list of what is not unique, and claim U1 now names only the combination, with the observation that would refute it.

Which sources count

Rank	Source	Highest grade it gives	Take care
1	The thing itself: code at a named commit, an experiment you ran, a run log	Verified	Give the commit or the command, so anyone can repeat it
2	The vendor's own documentation, terms and pricing pages	Verified, with the date read	Prices and terms move; a price without its date is Reported
3	A paper with its data or code	Verified for counts you can re-derive from it	Quote counts, not percentages: CooperBench states its gap three ways

Rank	Source	Highest grade it gives	Take care
4	A project's README, design record, release notes or issues	Verified for what the project says of itself; Reported for what it says works	gh-aw's design record for its queue is still a draft
5	Blogs, talks, analyst reports, search summaries, a model's answer	Reported	A lead to the primary source, never the source; this includes the deep-research report until its links are opened

When a source is out of reach

This project's sessions cannot open several hosts (section 2). Read GitHub's documentation from the github/docs repository's content files on raw.githubusercontent.com, at a commit you name. Read a paper from its authors' repository where there is one, as CooperBench's PDF was read from cooperbench/website. Leave the rest to claude.ai Research (section 3), then have a thread open each link the report gives. A claim whose only source stays out of reach keeps the grade Reported, with a note of what to re-read.

5. Placing any tool in the taxonomy

Use this for any system someone names as an alternative, a competitor or a partner.

The procedure

1. **Pin the version.** Name the repository and commit, or the product, plan and date. A placement goes stale when the tool ships.
2. **Find its claim.** Ask one question of the code, not the README: when two of its agents want the same piece of work, what stops the second, and what frees a claim nobody finishes? Most of the differences in the field sit in that answer (axis 5).
3. **Put it in a quadrant.** Where does the coordination state live, and what does one more agent run cost as the tool is usually run? Those are the two axes of the spec's placement figure.
4. **Fill the worksheet below**, one cell per axis, each with its source. Not stated is an answer; a guess is not.

5. **Score it on the five mechanisms**, (a) to (e) of the spec's section 8: yes, partial (say what is missing) or no. Only a yes counts toward what would refute claim U1: four of the five, or (a), (c) and (d) together. A tool that reaches either goes to the spec before anything else.
6. **Add it to the spec:** a row in the placement table of section 4 if it shares a cell with Symphony or Beethoven, a row in the matching table of section 5, and a line in the prior-art table of section 8 for each mechanism it has.

The worksheet

Copy this table for each tool.

#	Axis	Question	Answer	Source
0	Identity	Name; repository and commit, or product and plan; date read		
1	Stack layer	Which of L0 to L5 does it occupy, and what does it sit on?		
2	Autonomy	While it works, does a person operate, collaborate, consult, approve or only observe?		
3	Control topology	Who decides who does what: one agent, an orchestrator, planners, peers, or a shared medium?		
4	Communication medium	How does one agent learn of another's work?		
5	Concurrency control	What stops a second agent taking the same work, and what frees a claim nobody finishes?		

#	Axis	Question	Answer	Source
6	Dependencies	Which does it manage: shared resource, prerequisite, transfer, usability, simultaneity, task and subtask?		
7	Execution environment	Where does an agent run?		
8	Trigger and lifetime	What starts a run, and what ends it?		
9	State and memory	Where does the durable record live?		
10	Cost model and payer	Who pays for inference and for compute, and under what licence?		
11	Verification	What decides that work is acceptable, and can it block a merge?		
12	Domain and maturity	What work is it for, and is it released, pre-release or a design?		
(a)	Atomic claim on the forge	Yes, partial or no		
(b)	Idle release by CI, work kept	Yes, partial or no		
(c)	Board and coordinator in CI, state in forge objects	Yes, partial or no		
(d)	People's sessions and CI agents under one rule, on a subscription	Yes, partial or no		
(e)	Shipped protocol code checked by interleavings and mutants	Yes, partial or no		

A worked example: unsorry

unsorry (agenticsnz/unsorry), a swarm of agents that proves goals in Lean, is the closest rival the search found. Read at commit 029d328b on 2026-10-15 (research/taxonomy/verify/03).

#	Axis	Answer	Source
1	Stack layer	L3 coordination on L4 GitHub, for an L5 domain	Inferred from the rows below
3	Control topology	A shared medium with self-claiming peers, and a dispatcher in Actions	queue-dispatcher.yml
4	Communication medium	Traces in git: claim files on the branch claims, queued/prove/* branches, files on main, pull requests	ADR-004
5	Concurrency control	A lock with a timer: a claim file pushed to one shared branch, first push wins, checked again after the push; deleted two hours after the claim was made	ADR-004 lines 14 and 25; ADR-072; reaper.yml; tools/gate_b/config.py line 11
7	Execution environment	Agents on volunteers' own machines; the board, dispatcher, reaper and about 15 janitors in Actions	swarm/agent.sh; .github/workflows
8	Trigger and lifetime	Volunteers start their agents; the reaper runs every 15 minutes	reaper.yml
9	State and memory	Branches, files on main and pull requests; no central server	ADR-004

#	Axis	Answer	Source
10	Cost model and payer	Each volunteer pays for inference (claude -p, Codex, Gemini); coordination runs on Actions minutes	swarm/agent.sh line 2685
12	Domain and maturity	Formal mathematics; claims in this form since 2026-06-10	ADR-004
(a)	Atomic claim	Partial: files on one shared branch, not a ref per item	
(b)	Idle release	Partial: a timer from the claim's own timestamp, not idleness; the work is not kept	
(c)	Board in CI	Yes	
(d)	One rule for sessions and CI agents	Partial: volunteers' runs only; no agent in Actions, and no subscription token there	
(e)	Checked protocol code	No	

One yes and three partials: unsorry does not refute U1, but it is why U1 and U2 are worded as narrowly as they are. Axes 2, 6 and 11 were not read, and stay blank until someone reads them.

6. Explaining the approach

The argument, in order

Keep this order for every audience. The need comes before the mechanism, so the listener knows what the mechanism is for; the limits come before anyone asks, so the rest is believed.

1. **The need.** Agents that share a codebase get in each other's way. In CooperBench, where each of two agents worked in its own copy and could only message the other, one GPT-5 agent passed 315 of 652 feature pairs where two agents passed 183. The failures were duplicated and contradictory work, not a lack of skill, and chat between the agents did not raise success; in a later, smaller ablation, pairs given a shared git remote or task list did better than one agent, on weak evidence (spec 7.1).
2. **The mechanism.** Make the forge the coordinator. A claim is the push that creates the branch claude/N, and git lets exactly one of two racing creations win, so no lock server is needed. A board, the release of idle claims and the landing order of overlapping work are GitHub Actions jobs over GitHub's own objects, and a required check decides what merges (spec 2 and 9.1).
3. **The economics.** On a public repository those jobs run on GitHub's free standard runners and make no model call. The Claude work bills a flat plan, not tokens, and there is nothing to host (spec 9.2 and 9.6).
4. **The evidence.** The protocol's shipped code was checked over 16.7 million states against an oracle written from its spec, with mutants that prove the checks bite, and found no violation. Building the harness and reviewing the fixes it led to found 19 defects: six are fixed in release 0.4.1, eleven on main but not yet released, one was settled in the spec, and one (F8) is open. What is not yet evidence: no full-scale benchmark of Symphony exists (spec 2; question R1).
5. **The limits.** Inference is still paid, and one plan's usage window is shared. The terms questions are open, the server enforces a claim only at its creation, and one vendor sets the price (spec 10).

For a short explanation, use three of the spec's twelve axes, the three the study rests on: who pays for compute and inference (axis 10), how a claim is kept (axis 5), and what decides that work is acceptable (axis 11).

For grant reviewers

Lead with the science, not the software: AI-run research fails in ways its paper cannot show (spec 7.2), so the checks belong inside the run. Then the coordination, as the laboratory's rule of one issue, one branch, one pull request.

Three sentences to adapt:

> Studies of AI agents find that two agents splitting work on one codebase, each in its own copy and talking only by message, succeed less often than one, because they duplicate and contradict each other's work, and that AI-run research fails in ways its paper cannot show. Our laboratory runs its agents under a claim protocol in which each piece of work is one issue, one branch and one pull request, and plans to turn reporting checklists such as CLAIM and REFORMS into gates checked over every run's record, so that a tool, not a model, decides whether a result may be reported. One such check is built today; the others are planned.

For this audience:

- No numbers from this project's coordination experiment until you say so. The hold is on the experiment; whether CooperBench's published counts go in is your call.
- Say, part by part, what is built and what is planned (the tables in spec section 2). Reviewers discount a plan presented as a result.
- Say the stack is a design: no Beethoven repository runs Symphony's code today (spec 6.5).
- Match the grant packet: Beethoven is your proprietary software, and the award funds none of it.
- Name the data-handling preconditions before a reviewer does: Wayne State's approval of AI tools, no BAA on a personal plan, and Remote Control's limits under HIPAA (spec risk 9).

For engineers

Lead with the mechanism and its failure modes; engineers trust an account that names its own holes.

> Symphony makes a GitHub repository the coordinator for Claude Code agents. A claim is the push that creates branch claude/N, and git lets exactly one of two racing creations win, so there is no lock server; the board, the release of idle claims and the order in which overlapping work lands are deterministic Actions jobs on GITHUB_TOKEN, and Claude runs only where judgment is needed. The shipped protocol library is checked over every interleaving of two workers, to a depth of eight steps on one issue and seven on two, against an oracle written from its spec, and its known gaps are written down.

Questions to expect, with honest answers:

Question	Answer	Spec
What stops a push onto someone else's claim?	On GitHub, only the claim's creation is enforced. A fast-forward or forced push onto an existing claim is accepted, since the recommended setup leaves claude/** out of every ruleset so that a release can rename the branch. The local push guard refuses forced pushes and leases onto another worker's claim, and on 2026-10-15 it was found to judge a lease by the session's own directory rather than the clone git -C names, so a lease onto another worker's claim passed from a directory whose clone had neither branch	Risk 12
What if a worker's clock lies?	Idleness is read from the committer date, which the pusher writes. A fix that reads GitHub's own record of the push is written but not merged (finding F8)	Risk 13
What binds first at scale?	On a public repository, not minutes. Probably GitHub's API limits, though nobody has measured it: 1,000 requests an hour on GITHUB_TOKEN, 100 queued runs, 20 concurrent jobs on Free, and no retry on 429 in the library yet	9.3; risk 5; R3
Isn't this gh-aw's work queue?	Its claim is also a compare-and-swap on a git ref, but a trusted scheduler grants claims to Actions runs only, a claim never expires for idleness, and its Claude engine refuses the subscription token	4; 5.1
How is it tested?	1,379 library tests, and a harness that drives the shipped library through every interleaving to a set depth (5 in the required check, 7 and 8 in the weekly deep searches) against an oracle from the spec's text, with 93 mutants	2

For lab leads

Lead with what people have to do and what it costs.

> Your lab's people and agents share one GitHub repository as their to-do list, their lock and their record: each task is one issue, one branch and one pull request, and there is no server to run. On a public repository the bookkeeping runs free on GitHub's machines; the Claude work is paid by each person's own plan, and agents that run inside GitHub share one plan's usage limits. A required check decides what merges.

What lab leads ask:

- **What does a new person need?** A GitHub login, and their own Claude plan for their own sessions.
- **What does it cost?** On a public repository, nothing in minutes. On a private one, a monthly quota and then a price per minute: in the spec's illustration, ten one-hour Claude runs a day come to about $90 a month on Team (spec 9.4). Claude plans are separate: Pro is $20 a month, Max $100 or $200.
- **Can it run our science?** Not on GitHub's free machines, which have no GPUs. The science runs on the cluster, which is Beethoven's part (spec 9.6).
- **Is it ready?** It is pre-release: 0.4.1 is the last release, and the rails for workers that run unattended, among them a cap on Claude runs and the qa check, are not built (spec risks 4 and 6).

Telling the GitHub Actions story honestly

Say	Do not say	Why
"Standard GitHub-hosted runners are free and unlimited on public repositories", quoting GitHub, with the date	Agents run for free	The minutes are free; inference is paid by a Claude plan (9.4)
Coordination steps make no model call and run on those minutes	Unlimited agents	20 concurrent jobs on Free, 1,000 API requests an hour, one plan's window (9.3)

Say	Do not say	Why
A public repository gets a 4 vCPU, 16 GB runner, twice a private one's	Symphony is free for its users today	Inference is paid by a Claude plan, a private repository bills minutes past its quota, and Symphony's own tests stay on private repositories until the Phase 3 rails exist (risk 4)
Rulesets, required checks, environments with reviewers, CodeQL and secret scanning are free there	Nothing else does this	The atomic claim, the serverless board and the free minutes all have prior art; only the combination is claimed (8)
Nothing to host: the forge is the lock, CI the scheduler, the repository the state	Beethoven runs on free GitHub minutes	It runs on the university cluster, and its repositories are private (9.6)
No other major hosted CI found offers new public projects unlimited free minutes	GitHub guarantees it	It is GitHub's price to change, and its terms tie runner use to the repository's own project (9.5; R5)
Anthropic documents the subscription token for claude-code-action	Anthropic approves agents on a subscription in CI	It advises an API key for a shared secret, and plan limits assume ordinary, individual usage (9.5; R6)

7. Combining Symphony and Beethoven with other software

Each recipe says what it gives, how it is put together, and whether it works today. None is a step to take now. Each one that is not built is a candidate issue, and the links use OWNER/REPO for the repository you mean.

7.1 Symphony with GitHub's own security gates

Works today on a public repository. It adds a free, deterministic second layer of checks on agents' code, which spec 7.2 shows is often insecure.

1. Open https://github.com/OWNER/REPO/settings/security_analysis (Settings, then Advanced Security in the sidebar's Security and quality section). Turn on Dependabot alerts, secret scanning with push protection, and CodeQL's default setup for code scanning.
2. Open https://github.com/OWNER/REPO/settings/rules (Settings, then Rules, then Rulesets) and edit the default branch's ruleset. Add the rule Require code scanning results beside the required check named check. The rule Require secret scanning alerts are resolved is in public preview and lists Secret Protection or Advanced Security as a prerequisite, so whether a public repository on GitHub Free can use it is Open.

Symphony's planned qa check would fold these into one required check with a report a person can read; it is not built. Verified for the rules from the github/docs repository at 8311902 (2026-10-15).

7.2 beethoven-qa as the Python lane of the qa check

Planned; a recorded decision. Contracts on every public function, searched by CrossHair with Z3, would run as one lane of the qa check, with its findings uploaded as SARIF so they appear beside CodeQL's. beethoven-qa is private and proprietary, so a public repository could run this lane only from a published gate library, which is an open question (spec 10).

7.3 Symphony's Claude-free core in a Beethoven repository

Waits on your decision. Beethoven forbids a stored Claude token in CI, and Symphony's plan says no Beethoven repository runs Symphony's code (spec 6.5). What could be shared is the core that needs no Claude token: the branch-name claim, the board, the protocol check, the push guard, the required check and auto-merge. Every route to that core runs Symphony's code: an adoption brings its workflows, and the plugin in symphony-marketplace brings the skills, /board and a push guard that is symphony-protocol's own code, though not the board's workflows or the protocol check (Verified, the plugin's README). So the step the plan allows is a Markdown-only pilot of the claim protocol in beethoven-qa, recommended after Symphony's Phase 3. If you change the rule instead, leave CLAUDE_CODE_OAUTH_TOKEN unset and register no worker in Actions.

7.4 Other coding agents under Symphony's claim

Needs building (Inferred). Any agent that can run git can claim: it pushes claude/N with the claim file as its first commit, adds the labels and opens a draft pull request that says Closes #N. Two things change outside Claude Code. The push guard is a Claude Code hook, so another agent loses that local fence and keeps only GitHub's. And Copilot's cloud agent and gh-aw name their own branches, so each needs an adapter: a deterministic job that claims claude/N before handing the issue over, with the agent's pull request opened from or merged into that branch. gh-aw's Claude engine would then bill an API key or Copilot credits, not the subscription.

7.5 GitHub's merge queue after Symphony's ordering

Not tried. Symphony's Overlaps section orders work that touches the same files; a merge queue then tests each pull request against the ones ahead of it, which catches conflicts that are not textual. The two are complementary. The merge queue is available in any public repository owned by an organization (Verified, github/docs at 8311902). A queue waits for the required checks on merge_group events, and the template's ci.yml runs only on pull_request, push and workflow_dispatch (ci.yml lines 70 to 75), so add a merge_group trigger first: without it, check may never reach the queue, and a queued merge then fails at the queue's timeout (Verified, github/docs at 8311902; whether ci.yml's push trigger would report check on the queue's branches is untested).

7.6 Workflow managers for Beethoven's outer loop

Chosen, not built. Metaflow is the chosen engine, but metaflow-slurm drives sbatch over SSH with a key file, which Beethoven's rule against touching a credential forbids unless the planned plain sbatch adapter replaces it. The alternatives are Snakemake with its Slurm executor plugin, Nextflow, Parsl and CWL; DVC can keep data versions, and MLflow or Weights and Biases the run records that the checklist gates would read (spec 6.1 and 6.3).

7.7 Actions as the control plane, the cluster as the compute plane

Works today in Beethoven's CI. GitHub's runners cannot reach the grid, so the gate runs on GitHub's runners and a grid lane on the cluster posts commit statuses named grid/... back to the pull request (spec 6.2). The same split lets any project keep its review and merge rules on GitHub while its heavy jobs run on hardware it already has.

8. Keeping the spec current

The spec is dated beside its title. A claim is only as current as the date its source was read, and prices, terms and neighbors are the first to drift.

What to re-check, and when

When this happens	Re-check	Spec sections
GitHub changes Actions prices, runner sizes, limits or terms	Pass 1, items 1 and 2; the github/docs and github/site-policy repositories	9.2 to 9.5; risk 1
Anthropic changes its plans, claude-code-action's sign-in, bare mode or its legal page	Pass 1, item 3	9.4 and 9.5; risks 2 and 3; R6
symphony-protocol publishes a release (0.5.0 is next)	Its release notes and validation/known.json	2 (maturity); risks 6, 12 and 13
A neighbor ships: gh-aw's queue leaves its draft design record, unsorry, Copilot's agents, openai/symphony	The worksheet in section 5, for that tool	4, 5 and 8
A research question meets its acceptance criteria: R1's full run, R3's measurement pass, R4 on the canary	The question's own criteria	7.1, 10 and 11
Beethoven builds a gate, or its launcher reaches a second university	The repository at its new commit	2 and 5.2; claims U4 to U6; risk 8
You lift the grant-data hold	Nothing to research; the grant reviewers' version in section 6 changes	7.1
Before any outside use: a submission, a talk, a public page	Pass 1 again if the spec's date is more than a month old (a default; change it if you like)	9, and every claim quoted

How to ask for an update

Post this in the project chat, with the link or the text of what changed:

```
Update the taxonomy spec: <what changed, with a link>. Re-grade the claims it touches, update the parts of the manual that rest on them, and tell me in the thread what changed.
```

The thread that makes the change writes its notes under research/taxonomy/notes/ (a new file for a new topic, or lines in the topic's file), puts adversarial checks under research/taxonomy/verify/, and moves the date beside the spec's title.

9. Glossary

Term	Meaning
Apptainer	A container runtime for clusters that runs without administrator rights
Blackboard	A shared store that independent agents read and write; the classic AI architecture (Nii, 1986)
Board	The file STATE.md on the branch board, which board.yml rewrites from the issues, branches, claim files and pull requests; nobody edits it by hand
Claim	The push that creates the branch claude/N for issue N, whose first commit adds a claim file naming the worker, the plan and the files to touch
CLAIM	The Checklist for Artificial Intelligence in Medical Imaging, updated in 2024 (Radiology: Artificial Intelligence)
claude-code-action	Anthropic's GitHub Action, anthropics/claude-code-action, which runs Claude Code inside a workflow
Compare-and-swap	An update that succeeds only if the value is still what the writer last saw; git applies one to a ref on every push

Term	Meaning
Coordinator	The deterministic steps in the board's run that release idle claims and announce overlaps, and Claude in coordinator.yml, which plans epics into tasks
CrossHair, Z3	A tool that searches by symbolic execution for inputs that break a Python function's contracts, and the solver it uses
Epic	An issue that asks for more than one piece of work, which the coordinator plans into tasks
Fast-forward	A push that only adds commits on top of the branch's tip, which git accepts without force
Fencing token	A number that rises with each grant of a lock, so a resource can refuse a holder whose lease has expired
Five-hour window, weekly limit	The usage limits of a Claude plan, shared by everything that runs on it
gh-aw	GitHub Agentic Workflows (github/gh-aw), which compiles agent workflows written in Markdown into Actions; its work queue is Symphony's closest neighbor
GITHUB_TOKEN	The token GitHub gives each workflow run, scoped to its repository, with 1,000 API requests an hour
icontract	A Python library for contracts (preconditions, postconditions, invariants) on functions
Interleaving	One order in which the steps of several concurrent workers can happen
Lease	In distributed systems, a lock that expires unless renewed (Gray and Cheriton, 1989). In git, --force-with-lease is a forced push that succeeds only if the remote branch is still where the pusher expects
Mutant	A copy of the code with one deliberate fault; a check that does not catch it does not bite

Term	Meaning
Open OnDemand	A web portal to a university cluster, through which Beethoven starts its jobs
Oracle	An independent statement of the right outcome, here written from the spec's text, against which every interleaving is checked
Overlap	Two pieces of open work that change the same file; the board lists them in the order they should land
PreToolUse hook	A command Claude Code runs before a tool call, which can refuse the call
Push guard	tools/guard_push.py, a PreToolUse hook that refuses forbidden pushes before they run, on the worker's own machine
REFORMS	Consensus recommendations for science that uses machine learning (Kapoor et al., Science Advances, 2024), 32 items
Release	What happens to a claim with no commit for 24 hours: its labels come off, a comment says what the branch holds, and the branch is renamed released/N-sha, so the work is kept and the name is free
Remote Control	Claude Code's feature that lets a session running on one machine be driven from claude.ai or the Claude app
Required check	A status check that a ruleset makes pass before anything merges; Symphony's is named check
Ruleset	GitHub's rules for branches or tags, such as required checks, blocked force pushes or a merge queue, set for a repository or an organization
Safe outputs	gh-aw's way of letting an agent ask for a write, such as a pull request or a comment, that a separate job with write permission carries out
Slurm, sbatch	The cluster's batch scheduler, and its command for submitting a job

Term	Meaning
Standard runner	A GitHub-hosted virtual machine of the default size; free and unlimited on public repositories
Stigmergy	Coordination through traces left in a shared environment, as Symphony's agents coordinate through branches and labels rather than messages
Subscription token	CLAUDE_CODE_OAUTH_TOKEN, made by claude setup-token, which lets claude-code-action bill a person's Claude plan instead of an API key
TRIPOD+AI	The reporting guideline for clinical prediction models that use regression or machine learning (BMJ, 2024)
Worker	Anything that claims issues: a person's sessions, a routine, or an agent in GitHub Actions, each labelled worker:name